

232. Implement Queue using Stacks Easy

Acceptance: 69.1% | [LeetCode](#)

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element `x` to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only `push to top`, `peek/pop from top`, `size`, and `is empty` operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

```
**Input**
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
**Output**
[null, null, null, 1, 1, false]

**Explanation**
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is
front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false
```

Constraints:

- $1 \leq x \leq 9$
- At most `100` calls will be made to `push`, `pop`, `peek`, and `empty`.
- All the calls to `pop` and `peek` are valid.

Follow-up: Can you implement the queue such that each operation is **amortized** $O(1)$ time complexity? In other words, performing `n` operations will take overall $O(n)$ time even if one of those operations may take longer.

Claude Opus Solution

Implement Queue using Stacks

1. Intuition

The core challenge: a **stack is LIFO** (Last In, First Out), but a **queue is FIFO** (First In, First Out). These are opposite orderings.

Key Insight: If you pop everything from one stack and push it into another stack, the order gets *reversed*. Reversing a LIFO order gives you FIFO order!

```
Stack A (push order 1,2,3):  [1, 2, 3]  <- top is 3
Pour into Stack B:          [3, 2, 1]  <- top is 1  ✓ Now front of queue is on top!
```

So we use **two stacks**: - `in_stack`: receives new pushes (newest elements on top) - `out_stack`: serves pops/peeks (oldest elements on top)

We only transfer elements from `in_stack` to `out_stack` when `out_stack` is **empty**. This lazy transfer is what gives us amortized $O(1)$.

2. Approach (Pseudocode)

```
push(x):
    Just append x to in_stack          # O(1)

_transfer():
    # Helper: only when out_stack empty
    if out_stack is empty:
        while in_stack not empty:
            move top of in_stack -> out_stack  # reverses order

peek():
    _transfer()                        # ensure oldest element is on top of out_stack
    return out_stack top
```

```

pop():
    _transfer()
    return out_stack.pop()           # remove & return oldest

empty():
    return both stacks are empty

```

3. Python Solution

```

class MyQueue:
    def __init__(self):
        # in_stack: elements are pushed here (newest on top)
        # out_stack: elements are popped/peeked here (oldest on top)
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        # Always push to in_stack. O(1).
        # We defer any reordering work until a pop/peek needs it (lazy).
        self.in_stack.append(x)

    def peek(self) -> int:
        # If out_stack is empty, we must transfer all elements from
        # in_stack to out_stack. This REVERSES the order, so the
        # oldest pushed element ends up on TOP of out_stack.
        if not self.out_stack:
            while self.in_stack:
                # pop() from in_stack gives newest first,
                # but each gets stacked, reversing into FIFO order.
                self.out_stack.append(self.in_stack.pop())
        # The top of out_stack is the front of our queue.
        return self.out_stack[-1]

    def pop(self) -> int:
        # Reuse peek() to guarantee out_stack has the front element on top,
        # then remove it. This avoids duplicating the transfer logic.
        self.peek()
        return self.out_stack.pop()

    def empty(self) -> bool:
        # Queue is empty only if BOTH stacks have no elements.
        return not self.in_stack and not self.out_stack

# Usage:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()

```

4. Complexity Analysis

Operation	Worst Case	Amortized
push	O(1)	O(1)
pop	O(n)	O(1)
peek	O(n)	O(1)
empty	O(1)	O(1)

Space Complexity: O(n) — we store n elements total split across two stacks.

Why is pop/peek amortized O(1)? (Answers the follow-up)

Although a single `pop` / `peek` can cost O(n) when it triggers a transfer, **each element is moved at most twice** over its entire lifetime: 1. Pushed onto `in_stack` (once). 2. Moved from `in_stack` to `out_stack` (once).

For **n operations**, total work is bounded by O(n). Spreading that cost across all operations gives **O(1) amortized** per operation. The expensive transfer "pays off" because subsequent pops are cheap O(1) until `out_stack` drains again.

5. Alternative Approaches

Approach A — Naive Single List (the `my_solution.py` above): Use one list; `pop` removes `elements[0]` . - ❌ This **violates the constraint** of using only stack operations (it accesses the front directly). - ❌ `pop` is $O(n)$ because `self.elements[1:]` rebuilds the list every time — *not* amortized $O(1)$.

Approach B — Costly Push (push-heavy two stacks): Keep the queue order maintained in one stack at all times. On every `push` : 1. Dump all of `stack1` into `stack2`. 2. Push the new element onto `stack2`. 3. Dump everything back into `stack1`.

- `push` : $O(n)$, but `pop` / `peek` : $O(1)$ (always ready).
- Better only if reads vastly outnumber writes. The lazy two-stack solution (our main one) is generally preferred because it achieves **amortized $O(1)$ for all operations**.

Recommendation: The two-stack lazy-transfer solution is the optimal, standard answer — it satisfies the follow-up and uses only valid stack operations.

Codex Solution

1. Intuition

A stack is **LIFO**: last in, first out.

A queue is **FIFO**: first in, first out.

To simulate FIFO behavior using stacks, we use **two stacks**:

- `in_stack` : used for newly pushed elements.
- `out_stack` : used for popping/peeking elements in correct queue order.

Key idea:

- When we push elements, we put them into `in_stack` .
- When we need to pop or peek from the front of the queue:
- If `out_stack` already has elements, its top is the queue front.
- If `out_stack` is empty, move all elements from `in_stack` to `out_stack` .

This reverses the order, making the oldest pushed element appear on top of `out_stack` .

Example:

```
push(1), push(2), push(3)
```

```
in_stack = [1, 2, 3]
out_stack = []
```

Transfer all from `in_stack` to `out_stack`:

```
in_stack = []
out_stack = [3, 2, 1]
```

Now top of `out_stack` is 1, which is the front of the queue.

This gives us amortized $O(1)$ operations.

2. Approach

Pseudocode

```
class MyQueue:
    initialize:
        in_stack = empty stack
        out_stack = empty stack

    push(x):
        push x onto in_stack

    peek():
        if out_stack is empty:
            while in_stack is not empty:
```

```

        pop from in_stack and push onto out_stack

    return top of out_stack

pop():
    call peek() to ensure out_stack has the front element
    pop and return top of out_stack

empty():
    return true if both stacks are empty

```

3. Python Solution

```

class MyQueue:
    def __init__(self):
        """
        Initialize two stacks.

        in_stack:
            Stores newly pushed elements.
            Elements are in normal insertion order.

        out_stack:
            Stores elements in reversed order.
            The top of out_stack is the front of the queue.
        """
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        """
        Push x to the back of the queue.

        Since new elements should be removed later than older elements,
        we simply push them onto in_stack.
        """
        self.in_stack.append(x)

    def pop(self) -> int:
        """
        Remove and return the front element of the queue.

        The front element is always on top of out_stack.
        If out_stack is empty, peek() will transfer elements from
        in_stack to out_stack first.
        """
        self.peek()
        return self.out_stack.pop()

    def peek(self) -> int:
        """
        Return the front element of the queue without removing it.

        If out_stack is empty, we move all elements from in_stack
        to out_stack. This reverses their order, so the oldest element
        becomes the top of out_stack.
        """
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())

        return self.out_stack[-1]

    def empty(self) -> bool:
        """
        Return True if the queue has no elements.

        The queue is empty only when both stacks are empty.
        """
        return not self.in_stack and not self.out_stack

```

4. Complexity Analysis

Let n be the number of elements in the queue.

`push(x)`

```
self.in_stack.append(x)
```

- Time complexity: $O(1)$
- Space complexity: $O(1)$ extra for the operation

`pop()`

Worst case:

- If `out_stack` is empty, we may move all elements from `in_stack` to `out_stack`.
- This can take $O(n)$ for one operation.

Amortized analysis:

- Each element is pushed to `in_stack` once.
- Each element is moved from `in_stack` to `out_stack` once.
- Each element is popped from `out_stack` once.

So across many operations, each element is handled only a constant number of times.

- Amortized time complexity: $O(1)$
- Worst-case time complexity: $O(n)$
- Space complexity: $O(1)$ extra for the operation

`peek()`

Same reasoning as `pop()`.

- Amortized time complexity: $O(1)$
- Worst-case time complexity: $O(n)$
- Space complexity: $O(1)$ extra for the operation

`empty()`

Checks whether both stacks are empty.

- Time complexity: $O(1)$
- Space complexity: $O(1)$

Overall Space Complexity

The queue stores all elements inside the two stacks.

- Space complexity: $O(n)$

5. Alternative Approaches

Alternative 1: Make `push` expensive

Instead of transferring elements during `pop` or `peek`, we can maintain the queue order during every `push`.

For each new element:

1. Move all elements from the main stack to a temporary stack.
2. Push the new element.
3. Move everything back.

This makes:

- `push`: $O(n)$
- `pop`: $O(1)$
- `peek`: $O(1)$

However, this is less efficient for frequent pushes.

Alternative 2: Use a normal list as a queue

In Python, one might write:

```
self.elements.append(x)
self.elements.pop(0)
```

But `pop(0)` is $O(n)$ because all remaining elements must shift left.

Also, this does not satisfy the problem's requirement of using only stack operations.

Key Pattern

This problem demonstrates a common technique:

Use two stacks to reverse order only when necessary.

This lazy transfer strategy is what gives the solution amortized $O(1)$ performance.